

# **Continual Learning & Memory Models**

## **COMS 6998**

**Week #10: April 1, 2026**

**Paper title: Learning to Learn without Forgetting by Maximizing Transfer and Minimizing Interference**



Chaofan Shou



@Fried\_rice



+ AI + NEWS + TECH

**Claude Code leak exposes a Tamagotchi-style 'pet' and an always-on agent** / The more than 512,000 lines of leaked code appear to show unreleased features, instructions for Claude, and more.

by + Emma Roth

Mar 31, 2026, 6:24 PM EDT

The Verge

11 Comments (All New)

## Anthropic inadvertently leaks source code for Claude Code CLI tool

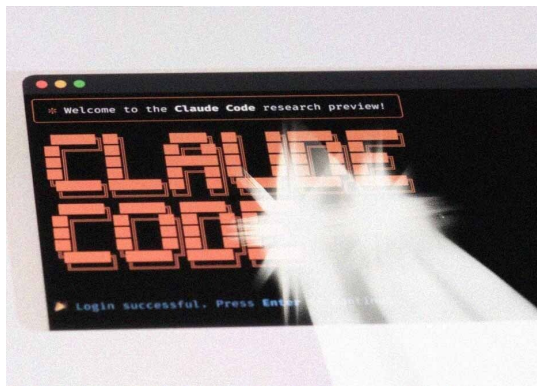
cybernews®

Published: 31 March 2026 · Last updated: 6 hours ago

**Repository unavailable due to DMCA takedown.**

This repository is currently disabled due to a DMCA takedown notice. We have disabled public access to the repository. The notice has been [publicly posted](#).

If you are the repository owner, and you believe that your repository was disabled as a result of mistake or misidentification, you have the right to file a counter notice and have the repository reinstated. Our help articles provide more details on our [DMCA takedown policy](#) and [how to file a counter notice](#). If you have any questions about the process or the risks in filing a counter notice, we suggest that you consult with a lawyer.



Claude code source code has been leaked via a map file in their npm registry!

Code: ...

a8527898604c1bbb12468b1581d95e.r2.dev/src.zip



sanat\_naft · 1d ago

Someone vibed too hard



469

```
shoudev claude-code % ls -alht cli.js.map
-rw-r--r--  1 shou staff  57M Mar 31 01:16 cli.js.map
shoudev claude-code % ls -alht src
total 2384
drwxr-xr-x  8 shou staff 256B Mar 31 01:17 endpoints
-rw-r--r--  1 shou staff 785K Mar 31 01:17 main.tsx
drwxr-xr-x  55 shou staff 1.7K Mar 31 01:17 .
drwxr-xr-x  10 shou staff 328B Mar 31 01:17 cli
drwxr-xr-x 183 shou staff 3.2K Mar 31 01:17 commands
drwxr-xr-x 331 shou staff 18K Mar 31 01:17 utils
drwxr-xr-x 146 shou staff 4.8K Mar 31 01:17 components
-rw-r--r--  1 shou staff 46K Mar 31 01:17 QueryEngine.ts
-rw-r--r--  1 shou staff 28K Mar 31 01:17 setup.ts
drwxr-xr-x  5 shou staff 168B Mar 31 01:17 server
drwxr-xr-x 13 shou staff 416B Mar 31 01:17 migrations
drwxr-xr-x 16 shou staff 512B Mar 31 01:17 keybindings
drwxr-xr-x  6 shou staff 192B Mar 31 01:17 skills
drwxr-xr-x  4 shou staff 128B Mar 31 01:17 plugins
-rw-r--r--  1 shou staff 22K Mar 31 01:17 dialogLaunchers.tsx
drwxr-xr-x  5 shou staff 168B Mar 31 01:17 screens
drwxr-xr-x 87 shou staff 2.7K Mar 31 01:17 hooks
-rw-r--r--  1 shou staff 56K Mar 31 01:17 interactiveHelpers.tsx
drwxr-xr-x 38 shou staff 1.2K Mar 31 01:17 services
-rw-r--r--  1 shou staff 3.4K Mar 31 01:17 replLauncher.tsx
-rw-r--r--  1 shou staff 617B Mar 31 01:17 costHook.ts
drwxr-xr-x  3 shou staff 96B Mar 31 01:17 moreright
drwxr-xr-x  3 shou staff 96B Mar 31 01:17 assistant
drwxr-xr-x  6 shou staff 192B Mar 31 01:17 zemate
```



**Chaofa**  
@Fried\_

just kidding!

sorry dario et al...

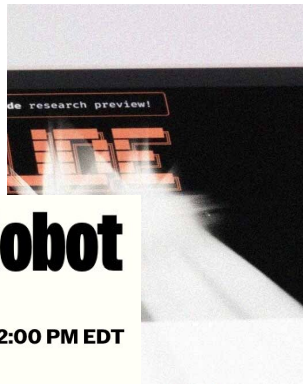
(pls dont send a giant destroyer robot after me)

' and an always-  
ode appear to  
d more.



ce code for

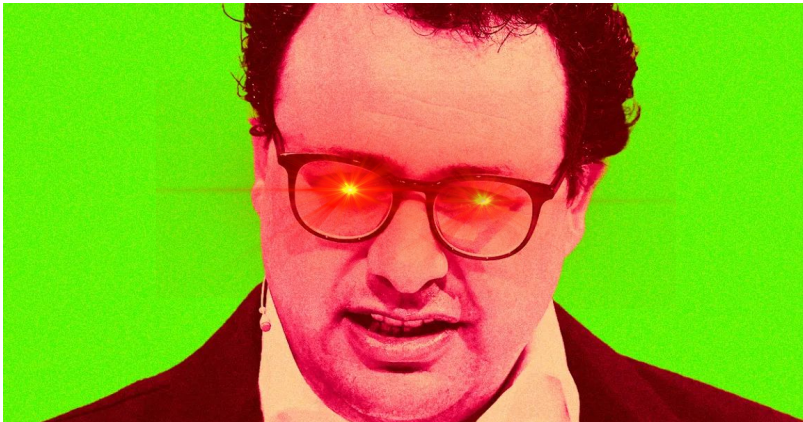
bernews®



Claude code  
been leaked  
their npm reg

Code: ...

a852789860  
d95e.r2.dev/



# The CEO of Anthropic Has a Framed Picture of a Giant Robot Destroying a City on His Wall



By Jon Christian / Published Jun 25, 2024 2:00 PM EDT

**Futurism**

notice, we suggest that you consult with a lawyer.

```

shoudev claude-code % ls -alht cli.js
-rw-r--r-- 1 shou staff 57M Mar 31 11:17 cli.js
shoudev claude-code % ls -alht src
total 2384
drwxr-xr-x  8 shou staff 256B Mar 31 11:17 src
-rw-r--r--  1 shou staff 785K Mar 31 11:17 src
drwxr-xr-x 55 shou staff 1.7K Mar 31 11:17 src
drwxr-xr-x 10 shou staff 320B Mar 31 11:17 src
drwxr-xr-x 183 shou staff 5.9K Mar 31 11:17 src
drwxr-xr-x 331 shou staff 10.7K Mar 31 11:17 src
drwxr-xr-x 246 shou staff 7.9K Mar 31 11:17 src
-rw-r--r--  1 shou staff 1.1K Mar 31 11:17 src
-rw-r--r--  1 shou staff 1.1K Mar 31 11:17 src
drwxr-xr-x  5 shou staff 160B Mar 31 11:17 src
drwxr-xr-x 13 shou staff 400B Mar 31 11:17 src
drwxr-xr-x 16 shou staff 448B Mar 31 11:17 src
drwxr-xr-x  6 shou staff 160B Mar 31 11:17 src
drwxr-xr-x  4 shou staff 112B Mar 31 11:17 src
-rw-r--r--  1 shou staff 1.1K Mar 31 11:17 src
drwxr-xr-x  5 shou staff 160B Mar 31 11:17 src
drwxr-xr-x 87 shou staff 2.8K Mar 31 11:17 src
-rw-r--r--  1 shou staff 1.1K Mar 31 11:17 src
drwxr-xr-x 38 shou staff 1.2K Mar 31 11:17 src
-rw-r--r--  1 shou staff 3.4K Mar 31 11:17 src
-rw-r--r--  1 shou staff 617B Mar 31 01:17 costHook.ts
drwxr-xr-x  3 shou staff 96B Mar 31 01:17 moreright
drwxr-xr-x  3 shou staff 96B Mar 31 01:17 assistant
drwxr-xr-x  6 shou staff 192B Mar 31 01:17 zemate

```

Oh.

# Stability-Plasticity → ??

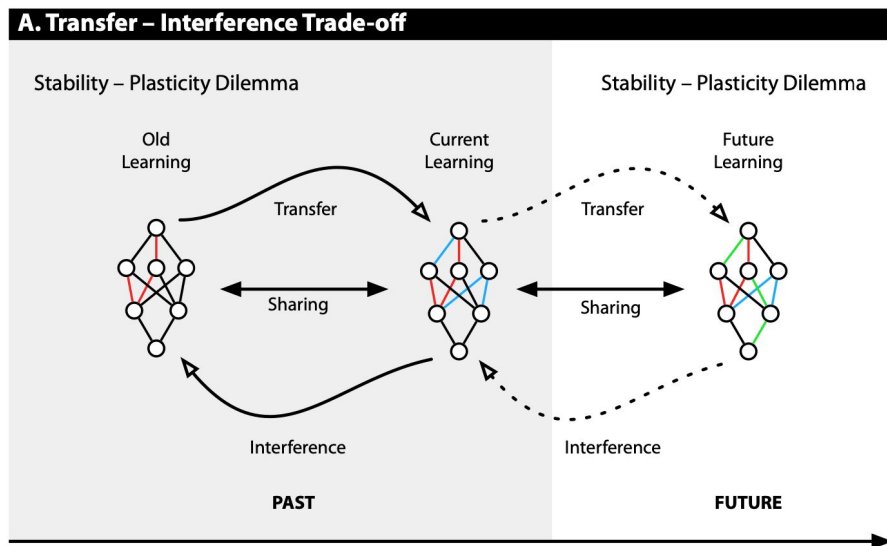
- **Problem (restating):** NNs assume **stationary i.i.d. data** BUT real-world agents face **continuous, non-stationary stream** of experiences → must learn incrementally while retaining what they've already learned
- Prior CL work frames the problem as **stability vs. plasticity**

|                                          |                                            |
|------------------------------------------|--------------------------------------------|
| <b>Stability:</b> don't forget old tasks | <b>Plasticity:</b> learn new tasks quickly |
|------------------------------------------|--------------------------------------------|

- **Elastic Weight Consolidation (EWC, [Kirkpatrick et al., 2017](#))**: penalizes changing "important" parameters (measured via Fisher info)
  - **Gradient Episodic Memory (GEM, [Lopez-Paz & Ranzato, 2017](#))**: stores examples per task, projects gradients to avoid interfering w/ past task memories, requires task boundaries
- Focus is **backward-looking** → protects past knowledge from current updates

# Stability-Plasticity → Transfer-Inference

- This paper argues we must also look **forward** (et al., 2018; Lee et al., 2018; Serrà et al., 2018). In this paper, we extend this view by noting that for continual learning over an **unbounded number of distributions**, we need to consider **weight sharing** and the **stability-plasticity trade-off** in both the **forward and backward directions in time** (Figure 1A).
- → **Transfer-Inference Tradeoff**: temporally symmetric view
  - Balancing BOTH **backward** AND **forward** directions
  - BOTH **avoid hurting old knowledge** AND **actively help future learning**



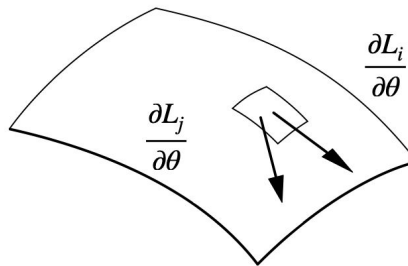


# How Do We Measure Transfer and Interference?

## TRANSFER (good)

When training on example  $i$  **improves** performance on example  $j$

$$\frac{\partial L(x_i, y_i)}{\partial \theta} \cdot \frac{\partial L(x_j, y_j)}{\partial \theta} > 0,$$

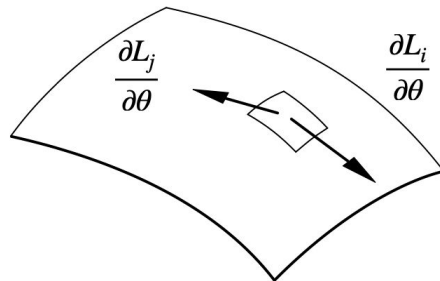


Gradient directions **ALIGN!**

## INTERFERENCE (bad)

When training on example  $i$  **hurts** performance on example  $j$

$$\frac{\partial L(x_i, y_i)}{\partial \theta} \cdot \frac{\partial L(x_j, y_j)}{\partial \theta} < 0.$$



Gradient directions **CONFLICT!**

**Same intuition as dot-product attention:**

$Q \cdot K$  = "how relevant is this key to this query?"

$\nabla L_i \cdot \nabla L_j$  = "how helpful is learning example  $i$  for example  $j$ ?"

# Method: Meta-Experience Replay (MER)

## Algorithm 1 Meta-Experience Replay (MER)

```
procedure TRAIN( $D, \theta, \alpha, \beta, \gamma, s, k$ )  
   $M \leftarrow \{\}$   
  for  $t = 1, \dots, T$  do  
    for  $(x, y)$  in  $D_t$  do  
      // Draw batches from buffer:  
       $B_1, \dots, B_s \leftarrow \text{sample}(x, y, s, k, M)$   
       $\theta_0^A \leftarrow \theta$   
      for  $i = 1, \dots, s$  do  
         $\theta_{i,0}^W \leftarrow \theta$   
        for  $j = 1, \dots, k$  do  
           $x_c, y_c \leftarrow B_i[j]$   
           $\theta_{i,j}^W \leftarrow \text{SGD}(x_c, y_c, \theta_{i,j-1}^W, \alpha)$   
        end for  
        // Within batch Reptile meta-update:  
         $\theta \leftarrow \theta_{i,0}^W + \beta(\theta_{i,k}^W - \theta_{i,0}^W)$   
         $\theta_i^A \leftarrow \theta$   
      end for  
      // Across batch Reptile meta-update:  
       $\theta \leftarrow \theta_0^A + \gamma(\theta_s^A - \theta_0^A)$   
      // Reservoir sampling memory update:  
       $M \leftarrow M \cup \{(x, y)\}$  (algorithm 3)  
    end for  
  end for  
  return  $\theta, M$   
end procedure
```

- **Step 1: Maintain a replay buffer  $M$** 
  - Fixed-size buffer of past examples
  - Updated via **reservoir sampling** (if new example arrives + buffer full  $\rightarrow$  randomly replace old example w/  $P(M_{\text{size}} / N)$ )
  - **Effect:** every example ever seen has **equal prob** of being in buffer (no recency bias, no task labels needed)
  - $\rightarrow$  approx sampling from full data distribution  $D$
- **Step 2: for each new example, build batches**
  - Draw  $s$  batches from  $M$ , each of size  $k$
  - **Every batch includes new example** to ensure we learn it
  - Remaining  $k-1$  examples sampled randomly from  $M$
- **Step 3: Train sequentially + Reptile meta-update**
  - Process each batch with SGD ( $\theta_0$  = before SGD,  $\theta_k$  = after)
  - Then **within-batch Reptile update (inner loop)**:  $\theta \leftarrow \theta_0 + \beta(\theta_k - \theta_0)$ 
    - ^ interpolation, NOT direct update  $\rightarrow$  direction implicitly contains  $\nabla$  alignment info (DP term) w/o 2nd  $\nabla$  computation
  - After all  $s$  batches: **across-batch Reptile update (outer loop)**
    - Same idea, but finds param direction that works for all batches
- **Step 4: Add new example to buffer** via reservoir sampling

# What Does MER Optimize?

**The MER learning objective:** In this work, we modify the **Reptile algorithm** to properly integrate it with an **experience replay module**, facilitating continual learning while **maximizing transfer and minimizing interference**. As we describe in more detail during the derivation in Appendix I, achieving the Reptile objective in an online setting where examples are provided sequentially is non-trivial and is in part only achievable because of our **sampling strategies for both the buffer and batch**. Following our remarks about experience replay from the prior section, this allows us to optimize for the following objective in a continual learning setting using our proposed MER algorithm:

$$\theta = \arg \min_{\theta} \mathbb{E}_{[(x_{11}, y_{11}), \dots, (x_{sk}, y_{sk})] \sim M} \left[ \underbrace{2 \sum_{i=1}^s \sum_{j=1}^k [L(x_{ij}, y_{ij})]}_{\text{"fit the data" (standard loss)}} - \underbrace{\sum_{q=1}^{i-1} \sum_{r=1}^{j-1} \alpha \frac{\partial L(x_{ij}, y_{ij})}{\partial \theta} \cdot \frac{\partial L(x_{qr}, y_{qr})}{\partial \theta}}_{\text{"align gradients across batches" (transfer-interference term from Slide 3)}} \right].$$

**Clever math trick:** gradient alignment term (2nd derivative) is never explicitly computed, emerges automatically from Reptile's sequential SGD + interpolation  
([Nichol et al., 2018](#))



# Validating the Framework: Does MER Align Gradients?

- **Setup:** For each new example, compute its gradient + gradients of 5 randomly sampled past examples (from a **separate** complete history, not the buffer) → measure mean dot product across all of training. Averaged over 5 runs.

| Model | MNIST Permutations | MNIST Rotations    | Many Permutations  |
|-------|--------------------|--------------------|--------------------|
| ER    | $-0.569 \pm 0.077$ | $-1.652 \pm 0.082$ | $-1.280 \pm 0.078$ |
| MER   | $+0.042 \pm 0.017$ | $+0.017 \pm 0.007$ | $+0.131 \pm 0.027$ |

Table 4: Analysis of the mean dot product across the period of learning between gradients on incoming examples and gradients on randomly sampled past examples across 5 runs on MNIST based benchmarks.

- **Standard Experience Replay (ER) alone:** all gradient dot products  $< 0$  → training on new examples actively **hurts** past performance
- **MER:** all  $> 0$  → learning new examples **helps** retain old knowledge
- **Direct empirical validation** of transfer-inference framework (Slide 3)
- Measured on a **separate complete history**, NOT the buffer itself (not circular)
- → **meta-learning creates alignment from how params are updated**  
→ **changes how the model learns from replayed data**

# Problem: The Transfer-Interference Trade-off

- Neural networks suffer from catastrophic forgetting when learning over non-stationary data
- Prior approaches focus only on backward stability (preserving old knowledge)
  - EWC: regularize weights deemed important for past tasks
  - GEM: project gradients so they don't interfere with stored memories
- Key insight: we must also consider forward transfer when learning new tasks
  - Reducing weight sharing to protect old knowledge can limit future learning
  - Need a temporally symmetric view: optimize for both past and future
- Transfer occurs when gradient dot product between examples is positive
- Interference occurs when gradient dot product is negative
- Goal: learn parameters that maximize gradient alignment across all examples

# Method: Meta-Experience Replay (MER)

- Combines two proven ingredients in a novel way:
  - Experience Replay: memory buffer with reservoir sampling to approximate i.i.d.
  - Reptile meta-learning: cheap first-order approximation of gradient alignment
- Core MER loop (for each incoming example):
  - Draw  $s$  batches of  $k$  examples from memory, interleaved with current example
  - Perform inner-loop SGD on each example individually (batch size = 1)
  - Apply Reptile meta-update within each batch, then across batches
- Effective objective (Eq. 7): minimizes task losses while maximizing pairwise gradient dot products between all sampled examples
- Avoids expensive second derivatives via Reptile's first-order Taylor approximation
- Computationally efficient: simple extension of SGD applied to a replay buffer

**Author:** Mihir Agarwal

# Key Results: Supervised & Reinforcement Learning

- MNIST Rotations / Permutations (20 tasks, buffer=5120):
  - MER: 89.56% / 85.50% retained accuracy vs. GEM: 87.58% / 83.02%
  - MER maintains strong learning accuracy while minimizing backward interference
- Gains increase as buffer shrinks (buffer=200):
  - MER: 77.42% / 73.46% vs. GEM: 67.38% / 55.42% (+10% advantage)
- Highly non-stationary settings (Many Permutations, Omniglot):
  - Omniglot: MER 75.23% vs. GEM 18.03% - nearly 4x improvement
  - GEM's quadratic program fails to converge; MER scales gracefully
- Continual RL (Catcher, Flappy Bird): DQN-MER retains earlier task performance while standard DQN forgets significantly

# Key Results:

| Model                         | MNIST Rotations         |                         |                         | MNIST Permutations      |                         |                         |
|-------------------------------|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
|                               | RA                      | LA                      | BTI                     | RA                      | LA                      | BTI                     |
| Online Independent Task Input | 53.38 $\pm$ 1.53        | 58.82 $\pm$ 2.17        | -5.44 $\pm$ 1.70        | 55.42 $\pm$ 0.65        | 69.18 $\pm$ 0.99        | -13.76 $\pm$ 1.19       |
| EWC                           | 60.74 $\pm$ 4.55        | 60.74 $\pm$ 4.55        | -                       | 55.80 $\pm$ 4.79        | 55.80 $\pm$ 4.79        | -                       |
| GEM                           | 79.98 $\pm$ 0.66        | 81.04 $\pm$ 0.34        | -1.06 $\pm$ 0.59        | 80.46 $\pm$ 0.80        | 81.20 $\pm$ 0.52        | -0.74 $\pm$ 1.10        |
| MER                           | 57.96 $\pm$ 1.33        | 78.38 $\pm$ 0.72        | -20.42 $\pm$ 1.60       | 62.32 $\pm$ 1.34        | 75.64 $\pm$ 1.18        | -13.32 $\pm$ 2.24       |
|                               | 87.58 $\pm$ 0.32        | 86.46 $\pm$ 0.46        | +1.12 $\pm$ 0.35        | 83.02 $\pm$ 0.23        | 80.46 $\pm$ 0.38        | <b>+2.56</b> $\pm$ 0.42 |
|                               | <b>89.56</b> $\pm$ 0.11 | <b>87.62</b> $\pm$ 0.16 | <b>+1.94</b> $\pm$ 0.18 | <b>85.50</b> $\pm$ 0.16 | <b>86.36</b> $\pm$ 0.21 | -0.86 $\pm$ 0.21        |

Table 1: Performance on continual lifelong learning 20 tasks benchmarks from [\(Lopez-Paz & Ranzato, 2017\)](#).

| Model | Buffer | MNIST Rotations         |                         |                         | MNIST Permutations      |                         |                         |
|-------|--------|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
|       |        | RA                      | LA                      | BTI                     | RA                      | LA                      | BTI                     |
| GEM   | 5120   | 87.58 $\pm$ 0.32        | 86.46 $\pm$ 0.46        | +1.12 $\pm$ 0.35        | 83.02 $\pm$ 0.23        | 80.46 $\pm$ 0.38        | <b>+2.56</b> $\pm$ 0.42 |
|       | 500    | 74.88 $\pm$ 0.93        | 85.90 $\pm$ 0.53        | -11.02 $\pm$ 1.22       | 69.26 $\pm$ 0.66        | 80.28 $\pm$ 0.52        | -11.02 $\pm$ 0.71       |
|       | 200    | 67.38 $\pm$ 1.75        | <b>85.40</b> $\pm$ 0.53 | -18.02 $\pm$ 1.99       | 55.42 $\pm$ 1.10        | 79.84 $\pm$ 1.01        | -24.42 $\pm$ 1.10       |
| MER   | 5120   | <b>89.56</b> $\pm$ 0.11 | <b>87.62</b> $\pm$ 0.16 | <b>+1.94</b> $\pm$ 0.18 | <b>85.50</b> $\pm$ 0.16 | <b>86.36</b> $\pm$ 0.21 | -0.86 $\pm$ 0.21        |
|       | 500    | <b>82.08</b> $\pm$ 0.31 | <b>85.66</b> $\pm$ 0.20 | <b>-3.58</b> $\pm$ 0.39 | <b>77.50</b> $\pm$ 0.46 | <b>83.90</b> $\pm$ 0.23 | <b>-6.40</b> $\pm$ 0.35 |
|       | 200    | <b>77.42</b> $\pm$ 0.78 | 83.02 $\pm$ 0.23        | <b>-5.60</b> $\pm$ 0.70 | <b>73.46</b> $\pm$ 0.45 | <b>84.42</b> $\pm$ 0.20 | <b>-9.96</b> $\pm$ 0.45 |

Table 2: Performance varying the buffer size on continual learning benchmarks [\(Lopez-Paz & Ranzato, 2017\)](#).

**Author: Mihir Agarwal**



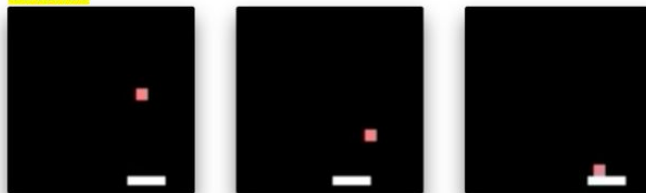
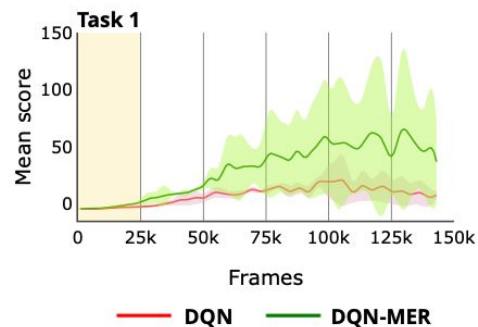
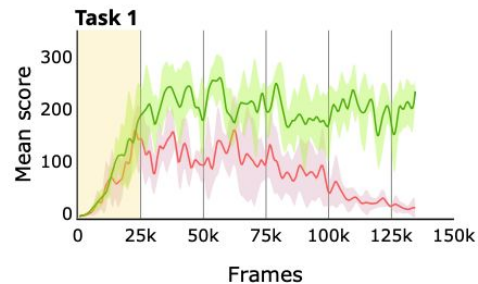
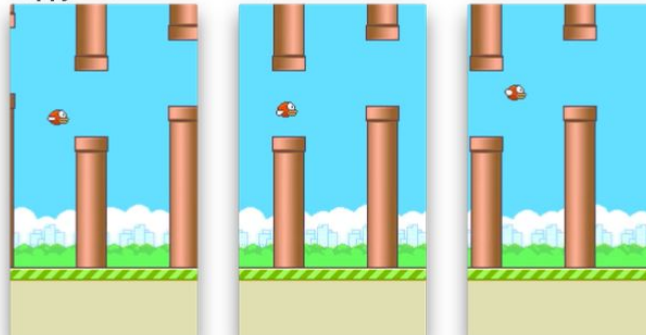
**Catcher****Flappy Bird**

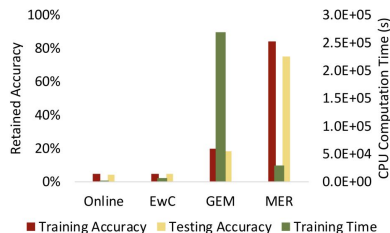
Figure 2: *Left*: a sequence of frames from **Catcher** and Flappy Bird respectively. The goal in **Catcher** is to capture the falling pellet by moving the racket on the bottom of the screen. In Flappy Bird, the goal is to navigate the bird through as many pipes as possible by making it go up or letting it fall. *Right*: average score in **Catcher** (above) and Flappy Bird (below) for evaluation on the first task which has slower falling pellets and a larger pipe gap.

# Analysis: Why MER Works

- Gradient alignment verified empirically (Table 4):
  - ER produces negative mean dot products (-0.57 to -1.65) = interference
  - MER shifts distribution to positive (+0.02 to +0.13) = transfer
- Ablation insights:
  - Experience replay with reservoir sampling alone outperforms GEM
  - Adding meta-learning on top consistently improves further
  - Meta-learning benefit grows as buffer size decreases
  - Adaptive optimizers (Adam, RMSProp) cannot replicate MER's gains
- Key contribution: a new conceptualization of continual learning as a temporally symmetric transfer-interference trade-off, not just backward forgetting
- Proactive (shapes weight space) vs. reactive (projects gradients after the fact)

## Strengths:

- Shifts away from GEM's ('dont make the past worse') to actually aiming to improve forward transfer (i.e. maximize alignment).
  - GEM rule:  $\mathbf{g}_{\text{new}} \cdot \mathbf{g}_{\text{old}} \geq 0$
  - MER objective: maximize  $\mathbf{g}_{\text{new}} \cdot \mathbf{g}_{\text{old}}$
- For settings with a large numbers of tasks (like Omniglot), MER is outperforms GEM both on accuracy and efficiency



- Clever mathematical workaround for avoiding solving the Hessian

## Weaknesses:

- This is not really “learning to learn”, it is just regularizing weight updates.
- No representation analysis - does it actually learn more general features than would be learned via GEM?
- Large cost for each new data sample
  - for each new incoming example, take several inner-loop gradient steps, then calculate the meta-update, THEN update weights.
- More hyper parameters than GEM makes it finicky - how to choose these numbers ex ante?
  - $\alpha$  : learning rate (actual step size per sample)
  - $\beta$  : Within-batch meta-learning rate (the leash pull back to starting point)
  - $\gamma$  : Across-batch meta-learning rate (the leash pull back to starting point across multiple updates)
  - $k$  : size of inner loop single batch (i.e. memories sampled per inner loop)
  - $s$  : number of outer loop steps (i.e. number of batches per new datapoint)

## Review score

- **Quality: 3/4** (Good) – Mathematical trick for computing the regularized loss is very clever.
- **Clarity: 3/4** (Good) – Theoretically framing is intuitive and well described.
- **Significance: 3/4** (Excellent) – Offers a very interesting shift from avoiding forgetting towards optimizing for transfer.
- **Originality: 4/4** (Excellent) – interesting an unique approach.

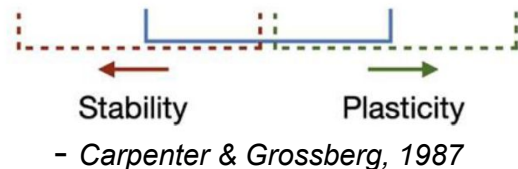
**Overall: 5 / 6 (Accept)**

**Reviewer: Max Bennett**

# The Landscape before MER

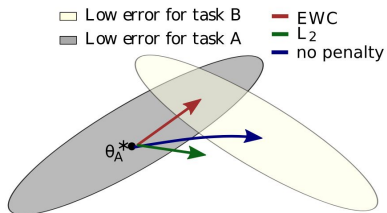
“Catastrophic Interference”

- McCloskey & Cohen, 1989



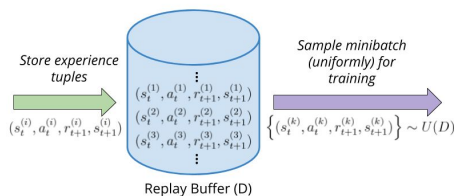
## Regularization-based

- EWC - Kirkpatrick et al., 2017



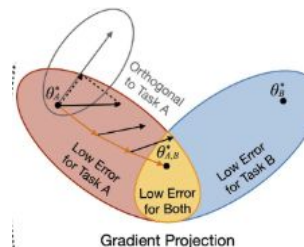
## Replay-based

- ER - Lin, 1992

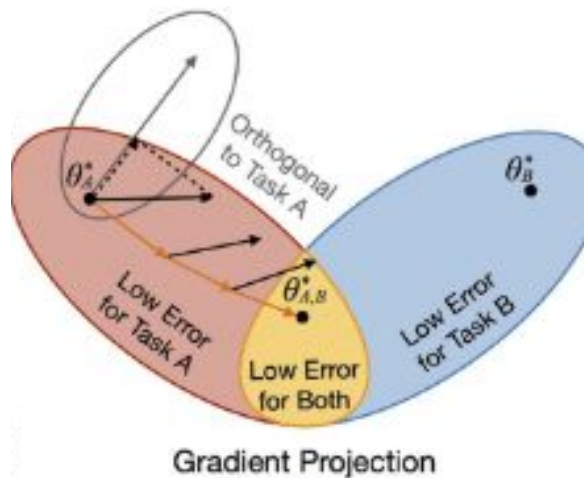


## Optimization-based

- GEM - Lopez-Paz & Ranzato, 2017



# Direct Ancestor: GEM



Core mechanism: (improve on current + old tasks)

$$\langle g, g_k \rangle := \left\langle \frac{\partial \ell(f_\theta(x, t), y)}{\partial \theta}, \frac{\partial \ell(f_\theta, \mathcal{M}_k)}{\partial \theta} \right\rangle \geq 0, \text{ for all } k < t.$$

- Memory buffer as constraint set, not as training data
- Most celebrated pre-MER approach, establishes the benchmarks that MER uses

| method                | 1 epoch    | 2 epochs   | 5 epochs   |
|-----------------------|------------|------------|------------|
| single, shuffled data | 0.83/      | 0.87/      | 0.89/      |
| single                | 0.53/-0.08 | 0.49/-0.25 | 0.43/-0.40 |
| independent           | 0.56/-0.00 | 0.64/-0.00 | 0.67/-0.00 |
| multimodal            | 0.76/-0.02 | 0.72/-0.11 | 0.59/-0.28 |
| EWC                   | 0.55/-0.19 | 0.59/-0.17 | 0.61/-0.11 |
| GEM                   | 0.86/+0.05 | 0.88/+0.02 | 0.89/-0.02 |

**Issue #1:** Expensive quadratic program

**Issue #2:** Reactive, learns nothing generalizable

**Archeologist:** Christian Montgomery

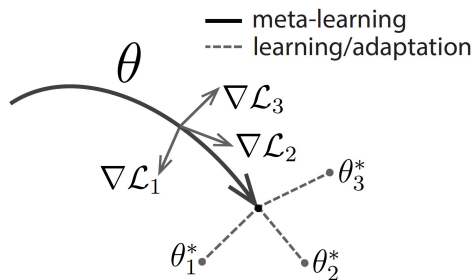


# Parallel Thread - MAML & Reptile

## Meta Learning: “Learning to Learn”

Question: Can we find weights that are good starting points for learning any task quickly?

**MAML** - Finn et al., 2017



- second derivatives (expensive)

Established that gradient alignment across tasks is a **learnable objective**

## On First-Order Meta Learning Algorithms (Reptile) - Nichol & Schulman 2018

---

**Algorithm 1** Reptile (serial version)

---

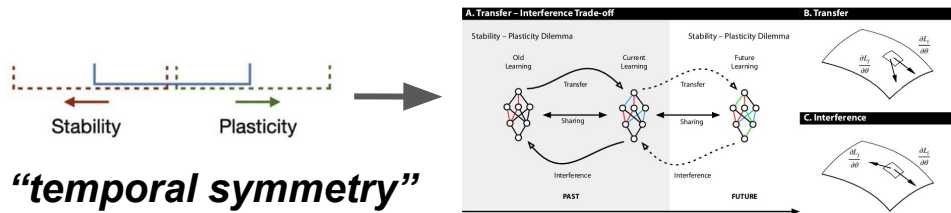
Initialize  $\phi$ , the vector of initial parameters  
**for** iteration = 1, 2, ... **do**  
    Sample task  $\tau$ , corresponding to loss  $L_\tau$  on weight vectors  $\tilde{\phi}$   
    Compute  $\tilde{\phi} = U_\tau^k(\phi)$ , denoting  $k$  steps of SGD or Adam  
    Update  $\phi \leftarrow \phi + \epsilon(\tilde{\phi} - \phi)$   
**end for**

---

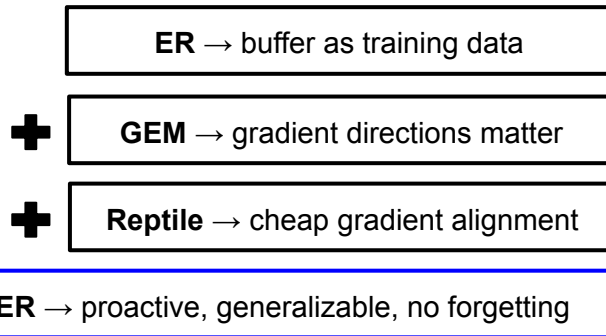
- sequential SGD + interpolation (cheap)

*Note: MER borrows this update rule, uses experience replay buffer vs. task batches*

# Synthesis: MER = Replay-based Infrastructure + Optimization-based Meta-learning



*“temporal symmetry”*



What was missing from the literature:  
Connected **gradient alignment** to a smooth learnable **objective function** for CL

MER says there’s a weight space where high transfer and low interference can both be achieved, not always a tradeoff

Proactive not reactive: shapes weight space so gradients tend to align, rather than project them after the fact (GEM)

Performance vs. existing approaches: Beats GEM and more efficient for small buffer sizes.

| Model | Buffer | MNIST Rotations |              |               | MNIST Permutations |              |               |
|-------|--------|-----------------|--------------|---------------|--------------------|--------------|---------------|
|       |        | RA              | LA           | BTI           | RA                 | LA           | BTI           |
| GEM   | 5120   | 87.58 ± 0.32    | 86.46 ± 0.46 | +1.12 ± 0.35  | 83.02 ± 0.23       | 80.46 ± 0.38 | +2.56 ± 0.42  |
|       | 500    | 74.88 ± 0.93    | 85.90 ± 0.53 | -11.02 ± 1.22 | 69.26 ± 0.66       | 80.28 ± 0.52 | -11.02 ± 0.71 |
|       | 200    | 67.38 ± 1.75    | 85.40 ± 0.53 | -18.02 ± 1.99 | 55.42 ± 1.10       | 79.84 ± 1.01 | -24.42 ± 1.10 |
| MER   | 5120   | 89.56 ± 0.11    | 87.62 ± 0.16 | +1.94 ± 0.18  | 85.50 ± 0.16       | 86.36 ± 0.21 | -0.86 ± 0.21  |
|       | 500    | 82.08 ± 0.31    | 85.66 ± 0.20 | -3.58 ± 0.39  | 77.50 ± 0.46       | 83.90 ± 0.23 | -6.40 ± 0.35  |
|       | 200    | 77.42 ± 0.78    | 83.02 ± 0.23 | -5.60 ± 0.70  | 73.46 ± 0.45       | 84.42 ± 0.20 | -9.96 ± 0.45  |

**Archeologist: Christian Montgomery**

## 2020: Some contrast: Dark Experience Replay

What if we don't need a meta-learning update like MER?

Keep a replay buffer but also store logits/outputs of previous inputs/labels

When replaying, force current model to match past predictions.

### Dark Experience Replay

Mixes buffer replay with knowledge distillation, what it calls “dark targets”

Targets general continual learning: tasks with blurred boundaries and shifting distributions

Positions itself and emphasises as a “strong baseline for future work”

## MER vs DER

Cites MER as “...replay cast as meta-learning” to max. transfer and min. interference

DER argues simpler rehearsal-style methods can be more robust and scalable

Shift towards strong, simple baselines

# 2024 Update: Continual Learning with Global Alignment

Forgetting is tied to gradient interference across tasks (opposing update directions)

Interference depends on correlations between hidden representations and “class vectors”

Uses gradient dot products as an analysis tool like MER!

Promotes “global alignment”: task representations are composed from shared pretrained token representations.

Ground cross-task correlations in pretrained semantics to help future, unseen tasks

Strong results without experience replay

## Ideas that Survived

Interference framed as gradient conflict (dot products / alignment)

“Future proofing” learning, which aims to reduce destructive updates while protecting the past

Alignment remains a design goal (now via representations)

## ... and didn't as much

Heavier training loop (replay + meta updates) vs simpler baselines that tested better

MER depended on storing/replaying examples, later work sought replay free or more scalable knobs

Early benchmarks were informative but not sufficient, community moved towards tougher, more realistic settings

# Can we scale MER to LLMs? How can we use LoRA?

## Scalability Bottleneck

- MER requires gradient dot products over the entire parameter space
- For a 7B-param model: dot product of two 7-billion-dim vectors per example pair (very scary)
- Reptile meta-updates require  $k$  sequential forward-backward passes per incoming example
- Computational cost:  $O(s \times k)$  full passes per example; can become prohibitive at LLM scale

## LoRA Insight

- LoRA constrains updates to low-rank subspaces (rank  $r = 8-64$ )
- Gradient dot products computed only over LoRA params; orders of magnitude smaller
- Low-rank structure may amplify alignment signal; less noise from irrelevant dimensions
- Pretrained base model provides strong initialization; gradient geometry is already structured

# Proposed Method: MER-LoRA

Apply MER's gradient alignment objective in the LoRA subspace during continual fine-tuning

- 1. Replay Buffer** Reservoir sampling of past task examples (text sequences)
- 2. LoRA-Space Alignment** Compute gradient dot products only over LoRA parameters
- 3. Reptile Meta-Update** Within/across batch updates on LoRA params; base model frozen

## Novel Research Questions at LLM Scale

**Layer-wise Alignment** — Do different transformer layers show transfer vs. interference for the same task pair?

**Rank Dynamics** — Can increasing LoRA rank provide more degrees of freedom for gradient alignment?

**Pretrained Priors** — Do pretrained representations already provide an alignment prior that MER can exploit?

**Buffer Efficiency** — Do information-rich text examples change MER's buffer size dynamics vs. MNIST digits?



# Experimental Plan & Broader Impact

## Experiments

**Setting:** Continual instruction tuning

Medical QA - Legal Summarization - Code Gen - Creative Writing

### Baselines:

- LoRA with naive sequential fine-tuning
- LoRA + experience replay (no meta-learning)
- LoRA + EWC-style regularization
- O-LoRA and other continual LoRA methods

**Key Analysis:** Replicate MER's Table 4 in LoRA space; measure gradient dot product distribution shift across layers

## Why This Matters

### Continual Alignment

Updating RLHF/DPO as user preferences shift without losing prior alignment

### Knowledge Updating

Incorporating new facts without degrading existing capabilities

### Multi-Task Agents

Tool-using agents that learn new skills sequentially without forgetting

### Adjacent Fields

Gradient alignment signal applicable to federated learning and curriculum design

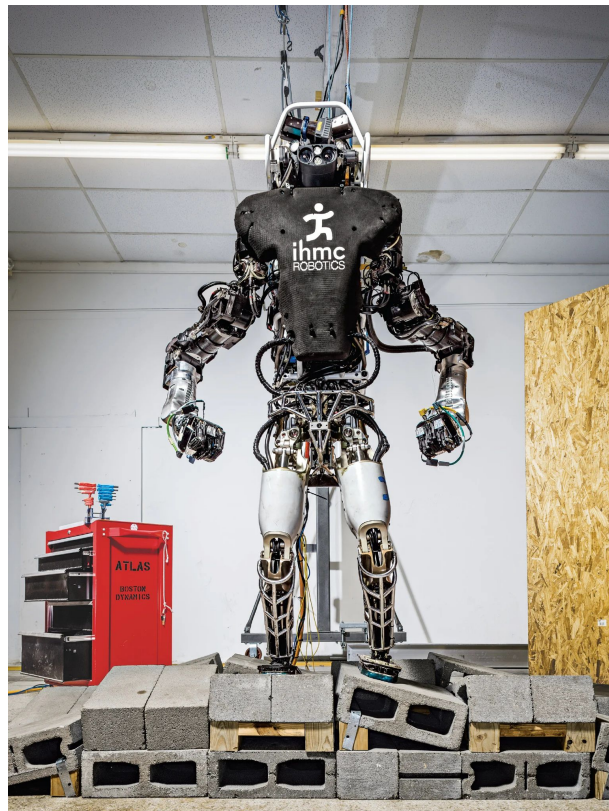
# A Reframing: MER as Meta-manifold learning

**Example of manifold learning:** learning the latent structure (manifold) of feasible poses of a robot so that it can reason about how to act given limitations of its hardware.

As environments and contexts changes the ability of the model to best learn from the environment without forgetting previous experiences changes. We may want to draw from different previous experiences depending on the current environment so that we can best draw similarities and contrasts, learning an optimal consensus.

**Example:** a robot tries to walk on concrete, gravel, sand, and deep snow. It might draw on past experiences when learning to walk on new substrates. These could better inform it how to update its general “walking” mechanism and improve balance in new settings (like over cinderblocks).

The manifold here is the robot’s mental model of its possible poses. As the model updates, the robot’s manifold updates to better facilitate walking in any environment.



**Researcher:** Dan Meyer (djm2261)

# Application to single cell foundation models

**CELL x GENE**: A rapidly growing repository for uniformly formatted single cell data advertising **~150M cells** across **2,083 datasets** and **1,106 cell types**.

**In order to learn effectively, need to generalize across many diverse experimental datasets.**

We can think about a foundation model that constantly updates as new datasets are deposited. Each new dataset may have different cell types, disease states, and sequencing technology.

The state of a cell can be thought of as its position on a manifold. Similar to poses of a robot.

**Goal:** We want to learn a latent space  $Z$  such that for all  $z = \text{encoder}(x)$  given any cell  $x$ :

- Local geometry reflects biologically meaningful neighborhood structure
- Global geometry/topology reflects broad cell-state organization
- The embedding generalizes across studies and technologies
- The representation remains useful as new datasets arrive.
- 

Allow for generalizations such as “How would cell type  $x$  react in patients with disease  $y$ ?”

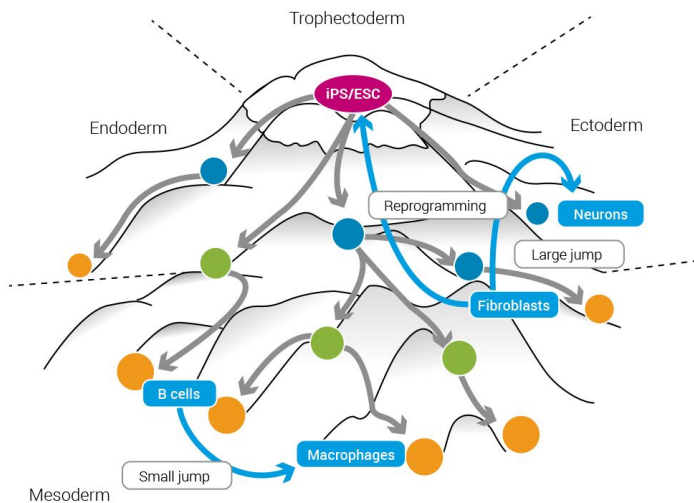
**Researcher:** Dan Meyer (djm2261)

# Application to single cell foundation models

**Goal:** We want to learn a latent space  $Z$  such that for all  $z = \text{encoder}(x)$  given any cell  $x$ :

- Local geometry reflects biologically meaningful neighborhood structure
- Global geometry/topology reflects broad cell-state organization
- The embedding generalizes across studies and technologies
- The representation remains useful as new datasets arrive.
- 

Allow for generalizations such as “How would cell type  $x$  react in patients with disease  $y$ ?”



**Researcher:** Dan Meyer (djm2261)

# Specific application to single cell foundation models

**With each new dataset we capture its inherent distribution shift, and cell type abundances compared to what has already been seen in the model's training dataset.**

We base our model off of **CONCORD**, a contrastive learning model for single cell integration that uses principled sample of similar and dissimilar cells from other datasets and the current dataset at both a local and global level.

In our proposed model, we:

- Structure batches that mix current and structured memory samples
- Perform updates on representation loss via MER/Reptile-style meta update
- Update memory based on geometric features of manifold and rarity of cell types

**What we are meta learning:**

- Which examples should influence each other (via replay / sampling)
- How gradients should be aligned across examples
- Which parts of the manifold should be preserved vs updated

# The Problem with Learning Multiple Skills at Once

Let's imagine you have too much time on your hands, so you've found yourself ...

- learning to play the **piano** ... and taking **tennis** lessons ... and teaching yourself **Spanish**.
- Practicing tennis → improves coordination → might help with piano (positive)
- Learning Spanish vocab → might get you mixed up with the Italian you learned last year (negative)

**Sometimes learning one skill helps with learning another ... and sometimes it gets in the way.**

How can we deal with this?

## Existing Landscape

- In the ML community, this problem has been primarily dealt with by trying to **preserve old knowledge** while learning new information.
- This is similar to saying “don’t forget your Italian” instead of trying to help learn Spanish more effectively.
- What if we delved deeper and asked “**when does learning one thing help vs hurt another?**”  
In response, the paper gives us two new concepts of:
  - Transfer → learning helps other skills
  - Interference → learning hurts other skills

So learning isn’t just about memorizing things.

It’s about how new information interacts with existing information.

## Key Insights

- Every time you learn something new, it slightly changes how you think.
- Sometimes that change is **helpful** (like tennis improving your coordination, which helps with piano). Other times, it creates **confusion** (like mixing up Spanish and Italian vocabulary).
- So when we learn something new, we can try to design the way in which we learn to **maximize this helpfulness** (“transfer”) and **minimize this confusion** (“interference”).
- Also, instead of just focusing on the effect of current learning with past knowledge, we can also try to **optimize for future learning** given current learning (maybe learning Spanish in some ways can help us with French in the future too while others might confuse us more).



# The Solution

- The goal is to maximize “transfer” and minimize “interference”
- Step 1: **Keep a Notebook** (Experience Replay)
  - occasionally review past material (old piano exercises, past vocabulary) to prevent forgetting
- Step 2: **Learn How to Learn** (Meta-learning)
  - everytime you learn something new, notice if it has made something you already know more confusing or not, and adjust your learning to keep important distinctions clear
- Step 3: **Do Both (Meta-Experience Replay)**

# Questions / Comments:

1. How does MER meta-update scale (computational overhead) to very big models with billions of parameters? The paper says it works well for smaller models like CNNs- does the algorithm scale well to SOTA very large models?
2. It seems like meta-learning is very computationally expensive since you're essentially calculating gradients of gradients (Reptile also calculates many inner and outer loops). How does the overhead compare to other techniques, such as Elastic Weight Consolidation (EWC)?

# Questions / Comments:

1. MER assumes an unbounded replay buffer containing raw past experiences to enforce gradient alignment — but in enterprise deployment contexts, client data is often subject to strict sovereignty and confidentiality constraints. If you can't replay raw experiences, does the gradient alignment mechanism break down entirely, or is there a bounded/privacy-preserving variant of MER that achieves comparable interference minimization?
2. The paper validates MER on MNIST variants where tasks are relatively homogeneous and sequentially distinct. In practice, enterprise deployments produce experiences that are semantically overlapping but contextually distinct — the same integration pattern with different client-specific constraints baked in. How does MER handle interference between tasks that are similar but not identical, where the gradient alignment signal may be ambiguous or actively misleading?

# Questions / Comments:

1. Does maximizing gradient alignment risk:
  - a. oversmoothing representations?
  - b. reducing task-specific specialization?

# Questions / Comments:

1. Does aligning gradients necessarily lead to better transfer? What makes the gradient a good representation that is used to optimize for better transfer?
2. The quality of the buffer will likely affect the performance of ER. How can a biased buffer be better handled?